Shaping Latent World Models via Latent Reasoning for Planning

ECE 285 Final Project · UC San Diego

Spring 2025

The Problem: Action-Conditioned World Modeling

Goal: Given (o_t, a_t), predict $\hat{o}_{t+1}$

$$(o_t,\ a_t) \xrightarrow{f_\theta} \hat{z}_{t+1} \xrightarrow{g_\phi} \hat{o}_{t+1}$$

The gap prior work leaves:

- Pixel-space models: no semantic structure
- Pure latent models: hard to decode faithfully
- LLM-style planners: output text, not pixels

Our middle ground:
A **structured latent token block** $\hat{z} \in \mathbb{R}^{K \times d}$ that is both semantically grounded and directly decodable to pixels.

Key Idea

$K = 8$ latent tokens bridge the predictor (Transformer) and decoder (U-Net).

The block is *fixed-length* $\Rightarrow$ easy to iterate for rollouts.

Why It Matters

- Enables multi-step planning
- Visual interpretability retained
- Compact: only 5.8 M params

Architecture: Four Components, One Pipeline

Vision Encoder E_v

4-layer CNN $\rightarrow$ M=64 patch tokens, d_v=64
Stride-2 convs: 3$\rightarrow$32$\rightarrow$64$\rightarrow$64 ch.

Action Encoder

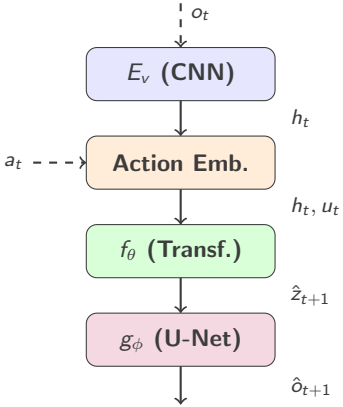
$\mathrm{Emb}(a_t) \in \mathbb{R}^{1\times 64}$, $a_t \in \{\mathrm{L,D,R,U}\}$

Latent Predictor f_θ (1.5 M)

4-layer Transformer, d=128, 4 heads
Output: $\hat{z}_{t+1} \in \mathbb{R}^{K\times d}$, K=8

Pixel Decoder g_ϕ (4 M)

U-Net + cross-attn bottleneck
$\hat{z}_{t+1} \rightarrow$ MHA $\rightarrow$ $\hat{o}_{t+1} \in \mathbb{R}^{3\times 64\times 64}$



Total: **5.8 M** params, trained from scratch

Three-Stage Training Pipeline

Stage 0 — Decoder Adapt.

What: Train $E_v + g_\phi$ as conditional autoencoder

Loss: $\mathcal{L} = \|g_\phi(o_t, \tilde{z}) - o_{t+1}\|^2$

where $\tilde{z} = \mathrm{pool}(E_v(o_t))$

Config: 20 ep., lr$=3\times10^{-4}$

Result: PSNR $= 31.3\,\mathrm{dB}$

Stage 1 — Format SFT

What: Train f_θ to match encoder targets

Loss:

$0.9\,\|\hat{z} - z^\star\|^2 + 0.1\,(1 - \cos)$

$z^\star = \mathrm{pool}(E_v(o_{t+1}))$

Config: 15 ep., lr$=1\times10^{-4}$

Result: cos-sim $= 0.997$

Stage 2 — Grounding

What: End-to-end pixel + embed loss

Loss:

$\mathcal{L}_{\mathrm{px}} + \lambda\,\|E_v(\hat{o}) - E_v(o^\star)\|^2$

$\lambda = 0.1$

Config: 30 ep., fp16, cosine LR

Result: PSNR $= 34.66\,\mathrm{dB}$

• Decoder learns domain → • Latent channel stabilized → • Full pipeline grounded

Experimental Setup: FrozenLake World Model

Environment

- FrozenLake-v1, 8×8, deterministic
- Custom 64×64 RGB renderer
- Colors: ice (blue), hole (black), goal (green), agent (orange)
- 4 discrete actions: L/D/R/U

Dataset

- 200 random map seeds
- 750 random-action rollouts per map
- ≈148K transitions total
- 160 / 20 / 20 maps → train/val/test

Hardware: Single NVIDIA L40S GPU

Why FrozenLake?

- Deterministic: error = model failure, not stochasticity
- Rich enough: 64-color spatial layout
- Fast: full pipeline in <3 h
- Controllable: ablations are clean

Evaluation

One-step: PSNR, SSIM, MSE, EmbedMSE

Multi-step rollout drift ($H = 5, 10, 20$):
$\text{Drift}_H = \frac{1}{H} \sum_k \|\hat{o}_{t+k} - o_{t+k}\|$

Main Results

Model	PSNR↑	SSIM↑	D_5↓	D_{10}↓
Stage 0 proxy	31.30	0.9318	0.00612	0.01041
Full (ours)	**34.66**	**0.9959**	**0.01524**	**0.02341**

- +3.36 dB PSNR over Stage 0 proxy
- SSIM = 0.9959 (near-perfect structural match)
- Drift_{10} = 0.02341 over 10 auto-regressive steps

What the Numbers Mean

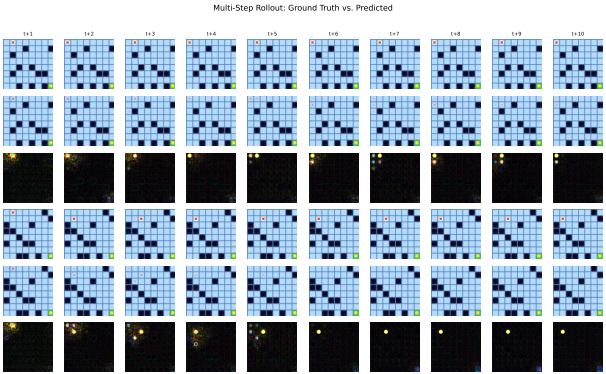
PSNR ≈35 dB: Near photo-quality; pixel errors barely visible at 64×64.

SSIM = 0.9959: Near-perfect structural similarity.

Drift_{10} = 0.02341: 65% lower than w/o embedding loss.

The embedding loss $\lambda\mathcal{L}_{embed}$ is key: penalizes drift in **feature space**, *not just pixels.*

Multi-Step Rollout Visualization



GT / **Pred** / $|\Delta|\times 10$
10 auto-regressive steps, no GT feedback.

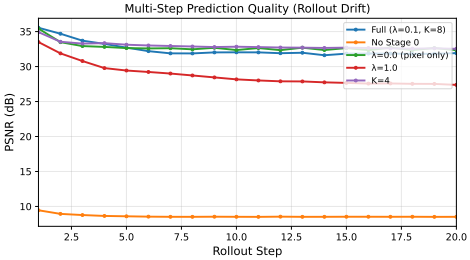
Key: Agent position and cell colors accurately tracked for steps 1–8. Errors concentrate near holes and boundaries.

Ablation Study

Config	PSNR	SSIM
Full model (30 ep.)	**34.66**	**0.9959**
No Stage 0	9.45	0.2112
$\lambda = 0$ (pixel only)	34.58	0.9965
$\lambda = 0.01$	34.52	0.9965
$\lambda = 1.0$	32.23	0.9918
$K = 2$	33.50	0.9946
$K = 4$	33.94	0.9953
$K = 16$	34.34	0.9960
Partial unfreeze	49.57	0.9999

λ: weight on embedding-space loss ($\lambda \mathcal{L}_{\text{embed}}$); controls feature-drift penalty.
K: number of latent tokens output by f_θ; larger K = richer representation.

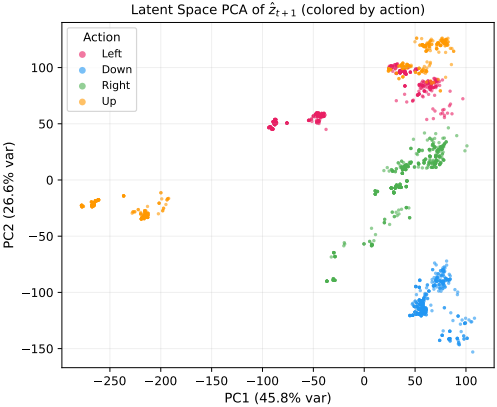
Partial unfreeze: decoder weights unfrozen in Stage 2 — overfits to memorize frames.



Takeaways:

- Stage 0 critical: ↓PSNR without it
- λ=0.1 optimal: embedding term stabilizes rollouts
- K=8 sweet spot (vs K=2/4/16)
- Partial unfreeze: degenerate (lookup table, not WM)

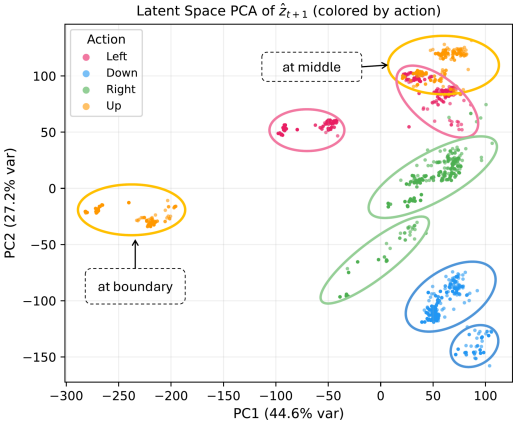
Latent Space Analysis



PCA of $\hat{z}_{t+1}$ tokens (2000 test samples, colored by action)

- Four clearly separated action clusters $\Rightarrow$ model learned action-conditioned latent structure
- Each cluster splits into **two sub-clouds**
- What do the sub-clusters mean? *(see next slide)*

Key Insight: Bimodal Latent Structure



Two sub-clusters per action

- **Boundary no-op**: agent at wall, action has no effect ($o_t = o_{t+1}$)
- **Effective transition**: agent moves, observation shifts by one cell

Why does this emerge?

The decoder must produce *different* pixel outputs for the two cases. This forces the latent predictor to encode whether the action is physically executable — **emergent, not supervised**.

≈24.7% of test transitions are boundary no-ops.

Key Insights

1. Embedding Loss = Stable Attractor

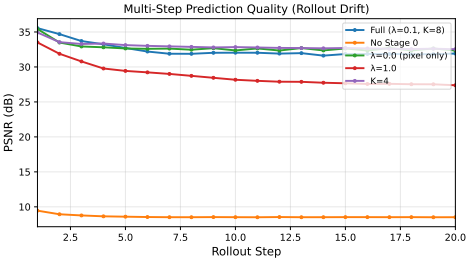
PSNR drops steeply t+1→t+8, then **plateaus** at ≈31.9 dB.
Without $\lambda\mathcal{L}_{\text{embed}}$: Drift_5 worsens **65%**.

2. Frozen Decoder = Inductive Bias

Unfreezing → 49.6 dB, but collapses to a **lookup table**.
Frozen decoder forces f_θ to learn a *general* world model.

3. Staged Training = Optimization Decomposition

Without Stage 0: catastrophic collapse to 9.45 dB (−25 dB).
Each stage narrows the search space for the next.



Full model (blue) plateaus; λ=0 drifts away; No Stage 0 collapses immediately.

Stage 3: World Model Enables Planning

Experiment: Does world model quality translate to downstream task performance?

Setup: 20 held-out maps $\times$ 5 episodes, max 80 steps

Method	Success↑	Avg Steps↓
Qwen2-VL-2B only	5%	14.0
Qwen2-VL-2B + WM	**5%**	**13.0**
Oracle BFS	100%	14.0

VLM+WM: lookahead consistency

1. **VLM** (LoRA fine-tuned) $\rightarrow$ action a
2. **WM** predicts frame for each of 4 actions
3. VLM re-queried from each predicted frame
4. Prune actions whose prediction elicits reversal
5. Return VLM's primary choice among survivors

finding

Both methods reach **5% SR** — tied.

Single-step look-ahead $\approx$ greedy search: cannot recover from local dead ends. The reversal filter is too weak to redirect an already-confused VLM.

Takeaway: WM quality (34.66 dB) $\neq$ automatic planning gains. Multi-step rollout + value function needed.

Conclusion

What we built:

- 3-stage pipeline: Decoder Adapt. $\rightarrow$ Format SFT $\rightarrow$ Grounding
- Latent token block $\hat{z} \in \mathbb{R}^{K \times d}$ as state carrier
- 5.8 M params, trained from scratch in <3 h

Key numbers:

- PSNR $= 34.66$ dB ($+3.36$ dB over baseline)
- SSIM $= 0.9959$, $\text{Drift}_{10} = 0.02341$
- Planning: VLM+WM 5% = VLM alone 5% (single-step look-ahead)

Takeaway: Staged training + embedding regularization enables stable, interpretable world modeling at low parameter cost.

Future: Scheduled sampling · stochastic envs · contrastive objectives

Staged Training = Decomposition

Stage 0: Visual prior established
Stage 1: Latent channel aligned
Stage 2: Pixel grounding

Without this: collapse to 9.45 dB.

Thank you! Questions welcome.